Tatiana Tomanek

ID 102041742

D206 PA

Data Acquisition

A1, Question Or Decision

What variables may cause a patient to stay in the hospital for extended periods of time?

A2, Required Variables

The variables in the data set includes patient identification variables:

- CaseOrder: (categorical) This variable is used to preserve the order of the original raw data file. Example - 1
- Customer_id: (categorical) An unique ID number given to each customer.
  Example - C412403
- Interaction: (categorical) Unique IDs given to each patient transactions, procedures, and admissions.
  Example - 8cd49b13-f45a-4b47-a2bd-173ffa932c2f
- UID: (categorical) Similar to the Interaction variable, unique IDs given to patient transactions, procedures, and admissions.
  Example - 3a83ddb66e2ae73798bdf1d705dc0932

Variables that represent the patient's demographic information:

- City: (categorical) The city that patient lives in, based on their billing statement.
  Example - West Point
- State: (categorical) The state that the patient lives in, based on their billing statement.
  Example - VA
- Country: (categorical) The county that the patient lives in, based on their billing statement. Example - Minnehaha
- Zip: (categorical) The zip code that the patient lives in, based on their billing statement.
  Example - 32446
- Lat: (categorical) The latitude coordinate that the patient lives at, based on their billing statement.
  Example - 34.34960
- Lng: (categorical) The longitude coordinate that the patient lives at, based on their billing statement.
  Example - -86.72508
- Population: (qualitative) The total population within a mile of the patient's home, based on census data.
  Example - 2951
- Area: (categorical) The type of area that the patient lives in, based on unofficial census data
  Example - Suburban

- TimeZone: (categorical) The time zone that the patient lives in, based on the patient's sign-up information
  Example - America/Chicago
- Job: (categorical) The patient's job, based on the admissions information
  Example - Aid worker
- Children: (qualitative) The number of children within the patient's home, based on the admissions information
  Example – 2
- Age: (qualitative) The patient's age, based on the admissions information
  Example - 18
- Education: (categorical) The highest level of education that the patient has earned, based on the admissions information
  Example - Bachelor's Degree
- Income: (qualitative) The annual income of the patient / primary insurance holder, based on the admissions information
  Example - 154.08
- Marital: (categorical) The marital status of the patient, based on the admissions information
  Example - Divorced
- Gender: (categorical) The patient's self-identification gender
  Example - Male

Variables that describe the patient's medical conditions and information:

- ReAdmis: (categorical) Identifies weather the patients have been readmitted within a month or not
  Example – Yes, No
- VitD_levels: (qualitative) The vitamin D level of the patient, measured in ng/mL
  Example - 18.99463952
- Doc_visits: (qualitative) The number of time that primary physician has visited the patient during their time at the hospital
  Example - 6
- Full_meals_eaten: (qualitative) The number of full meals that the patient ate while at the hospital
  Example - 0
- VitD_supp: (qualitative) The number of vitamin D supplements that the patient was given
  Example - 4
- Soft_drink: (categorical) Identifies weather the patient regularly drinks
  Example – Yes, No
- Initial_admin: (categorical) This describes how the patient was initially admitted into the hospital
  Example - Elective Admission

- HighBlood: (categorical) Describes if the patient has high blood pressure
  Example - Yes, No
- Stroke: (categorical) Describes if the patient has had a stroke
  Example - Yes, No
- Complication_risk: (categorical) Tells the risk of complications for the patient
  Example - High
- Overweight: (categorical) Describes if the patient is considered overweight
  Example – 0, 1
- Arthritis: (categorical) Describes if the patient has arthritis
  Example - Yes, No
- Diabetes: (categorical) Describes if the patient has diabetes
  Example - Yes, No
- Hyperlipidemia: (categorical) Describes if the patient has hyperlipidemia
  Example - Yes, No
- BackPain: (categorical) Describes if the patient has chronic back pain
  Example - Yes, No
- Anxiety: (categorical) Describes if the patient has an anxiety disorder
  Example – 0, 1
- Allergic_rhinitis: (categorical) Describes if the patient has allergic rhinitis
  Example - Yes, No
- Reflux_esophagitis: (categorical) Describes if the patient has reflux esophagitis
  Example - Yes, No
- Asthma: (categorical) Describes if the patient has asthma
  Example - Yes, No
- Services: (categorical) The main service the patient got while at the hospital
  Example - MRI
- Initial_days: (qualitative) How many days the patient initially stayed at the hospital
  Example - 1
- TotalCharge: (qualitative) The average amount the patient was charged daily
  Example - 4214.905346
- Additional_charges: (qualitative) The average amount charged for additional treatments, procedures, medicines, etc. to the patient
  Example – 12993.437350

The final variables represent responses, on a scale from 1 to 8, to a survey on various factors:

- Item1: (categorical) The patient's response to the hospital's timely admission.
  Example - 1
- Item2: (categorical) The patient's response to the hospital's timely treatment.
  Example – 2
- Item3: (categorical) The patient's response to the hospital's timely visits.
  Example -3
- Item4: (categorical) The patient's response to the hospital's reliability.

Example – 4

- Item5: (categorical) The patient's response to the hospital's options.
  Example - 5
- Item6: (categorical) The patient's response to the hospital's hours of treatment.
  Example – 6
- Item7: (categorical) The patient's response to the hospital's courteous staff.
  Example – 7
- Item8: (categorical) The patient's response to the active listening from doctor
  Example – 8

B1, Plan to Assess Quality of Data

To check the quality of the data, I needed to look at the number of duplicates, null values, and outliers. For duplicates, I will use .duplicated() and .unique(). For missing values, I will be using the .isnull.sum() function to find the variables with null values and how many there are. For outliers, I will use plt.hist(), sns.boxplot(), and .describe(). Next, I will use .unique to find the categorical variables that need to be re-expressed. (GeeksforGeeks) Finally I will use the .rename command to remane variable to make them more intuitive (Sentry)

B2: Justification of Approach

I started with duplicates, using the .duplicated() and .unique() functions to find any exact duplicates and any categorical values that are the same but formatted differently. To find the null values, I used the .isnull.sum() function to see the number of null values within each variable. I used a histogram and boxplot to help visualize the outliers. Additionally, I used the .describe() function to get an idea of the statistical summary of the variables. I used the preprocessing.LabelEncoder() and label_encoder.fit_transform commands from sklearn to help re-express the necessary variables. Finally the .rename was used to the last 8 variables to make them easier to understand.

B3: Justification of Tools

The programming language that I decided to use is Python. I have less experience in Python and found this to be a great opportunity to learn and better understand it. Additionally, I found that Python is more applicable to the jobs within the field I would like to go into. Besides that, Python's syntax and vast amounts of libraries and packages make it both versatile in what it can do and easier to understand. For this project, I decided to use these packages:

- Pandas to import, create datasets, and rename variables
- NumPy to help manipulate and process the dataset
- Missingno to make it easier to find missing values
- Matplotlin.pyplot to help label graphs
- Scipy.stats to find statistical values

- Seaborn to help make box plots and the PCA
- Preprocessing from sklearn to help re-express variables
- PCA from klearn.decomposition to help conduct the PCA

C1: Cleaning Findings

I found that this dataset had no duplicate inputs. For missing values, I found that Children had 2588, Age had 2414, Income had 2464, Soft_Drink had 2467, Overweight had 982, Anxiety had 984, and Initial_days had 1056 null values. While looking at the outliers, I found that the only variable with unexpected or unreasonable outliers was the VitD_Levels, with 500 observations above 40 ng/mL. These observations were well above the boxplot maximum. I found that the ReAdmis, HighBlood, Stroke, Arthritis, Diabetes, Hyperlipidemia, BackPain, Allergic_rhinitis, Reflux_esophagitis, and Asthma variables needed to be re-expressed.

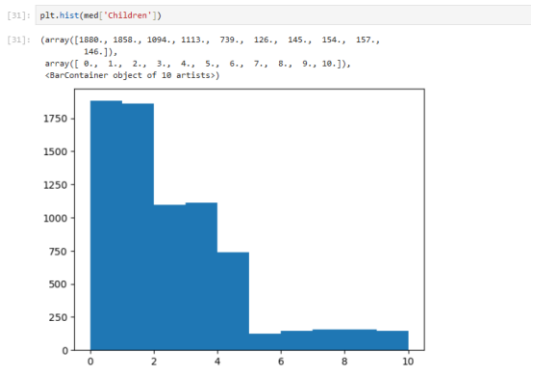
C2: Justification of Mitigation Methods

Since I did not find any duplicates, I did not need treatment for that. Next, I decided to fill in the missing values because there were too many nulls in too many variables to justify deleting either of them. I then split the column with missing values into numerical and categorical groups. For the numerical values, I started by creating histograms and seeing how the data was distributed, then filled the missing values with either their average or median. The variables Children and Income were both positively skewed, so I filled each with their medians: 1 child and $33942.28, respectively. The Age variable was uniform, so I filled in the null values with its mean, 53 years old. Finally, there was Initial_Days, which was bimodal, which means I could fill the null values with either the median or the mode. I decided to use the median of 34 days because it had the least statistical change. For the categorical data, I decided to fill all the nulls with the new variable 'None'. I did this because I felt that filling the nulls with anything else would mess up the data too much. Then, I moved on to finding the outliers. The only variable that I decided to impute was the VitD_Levels, where I converted all outliers to the median of 17.935 ng/mL . I reexpressed the variables ReAdmis, HighBlood, Stroke, Arthritis, Diabetes, Hyperlipidemia, BackPain, Allergic_rhinitis, Reflux_esophagitis, and Asthma from "Yes" and "No" to "1" and "0". I found that the last 8 columns Item1- Item8 did not make sense so I renamed them to be TimelyAdmission, TimelyTreatment, TimelyVisits, Reliability, Options, HoursOfTreatment, CourteousStaff, and ActiveListening.

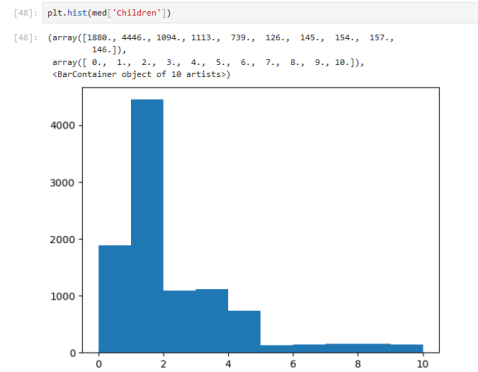
C3: Summary of the Outcomes

After implementing the changes to the missing values all had the same distribution and minimal to no changes to their statistics like the mean. Here you can see how filling each missing value in the numerical variables has affected their distribution at mean values:

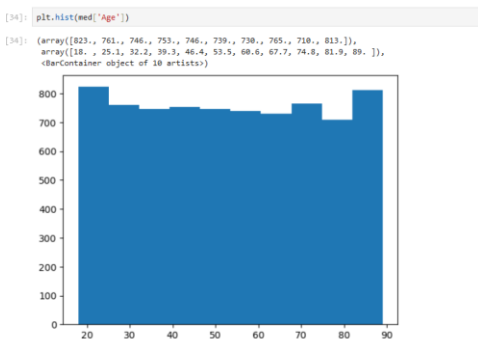
The Children variable kept its average of 2 children, and its histograms looked like this:



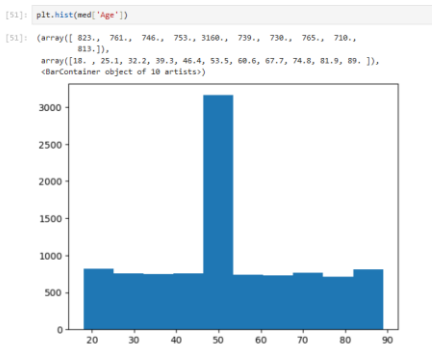
→



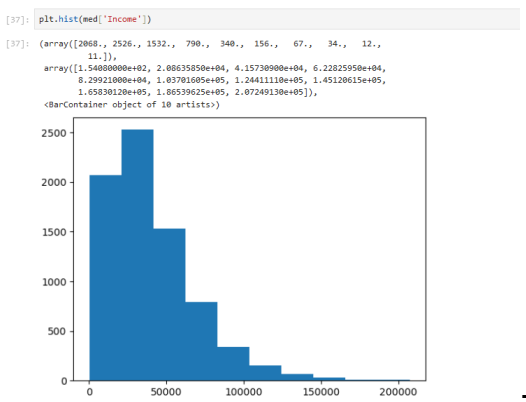
The Age variable kept its average of 53year and its histograms looked like this:



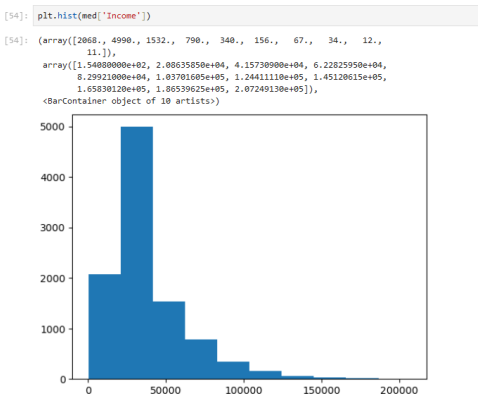
→



The income variable went from an average of $ 40,484.44 to $38872.45, and its histograms looked like this:



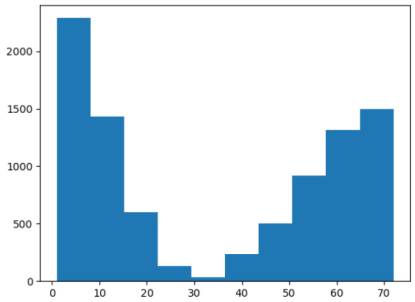
→



The Initial_days variable kept its average of 34 days, and its histograms looked like this:

```
[40]: plt.hist(med['Initial_days'])
```

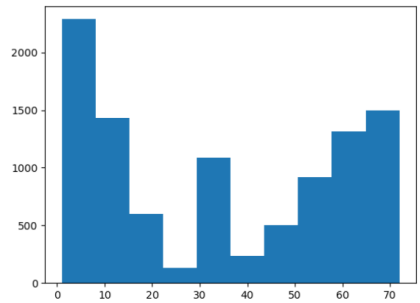
```
[40]: (array([2288., 1429.,  599.,  134.,   32.,  235.,  502.,  916., 1314.,
        1495.]),
 array([ 1.00198092,  8.09993146, 15.197882  , 22.29583253, 29.39378307,
        36.49173361, 43.58968415, 50.68763469, 57.78558522, 64.88353576,
        71.9814863 ]),
 <BarContainer object of 10 artists>)
```



→

```
[57]: plt.hist(med['Initial_days'])
```

```
[57]: (array([2288., 1429.,  599.,  134., 1088.,  235.,  502.,  916., 1314.,
        1495.]),
 array([ 1.00198092,  8.09993146, 15.197882  , 22.29583253, 29.39378307,
        36.49173361, 43.58968415, 50.68763469, 57.78558522, 64.88353576,
        71.9814863 ]),
 <BarContainer object of 10 artists>)
```



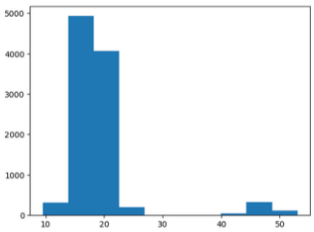
Then after filling in the qualitative there are no more nulls in the data set. Which can be seen here:

[63]:
```
med.isnull().sum()
#Final check for any missing variables
```

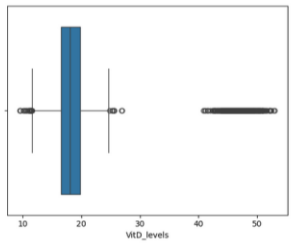
[63]:
```
CaseOrder             0
Customer_id           0
Interaction           0
UID                   0
City                  0
State                 0
County                0
Zip                   0
Area                  0
Timezone              0
Job                   0
Children              0
Age                   0
Education             0
Employment            0
Income                0
Marital               0
Gender                0
ReAdmis               0
VitD_levels           0
Doc_visits            0
Full_meals_eaten      0
VitD_supp             0
Soft_drink            0
Initial_admin         0
HighBlood             0
Stroke                0
Complication_risk     0
Overweight            0
Arthritis             0
Diabetes              0
Hyperlipidemia        0
BackPain              0
Anxiety               0
Allergic_rhinitis     0
Reflux_esophagitis    0
Asthma                0
Services              0
Initial_days          0
TotalCharge           0
Additional_charges    0
Item1                 0
Item2                 0
Item3                 0
Item4                 0
Item5                 0
Item6                 0
Item7                 0
Item8                 0
dtype: int64
```

For outliers the only cleaning I did was to VitD_levels which had a graph that went from this:

```
[75]: plt.hist(med['VitD_levels'])
[75]: (array([ 313., 4930., 4063.,  194.,    0.,    0.,    0.,   49.,  329.,
         122.]),
 array([ 9.51901164, 13.86902289, 18.21903414, 22.56904539, 26.91905665,
        31.2690679 , 35.61907915, 39.9690904 , 44.31910166, 48.66911291,
        53.01912416]),
 <BarContainer object of 10 artists>)
```

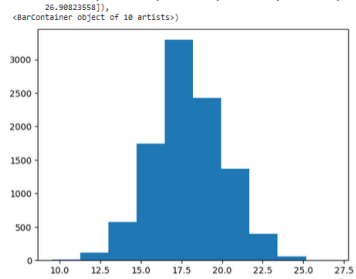



```
[76]: sns.boxplot(x = 'VitD_levels', data= med)
[76]: <Axes: xlabel='VitD_levels'>
```

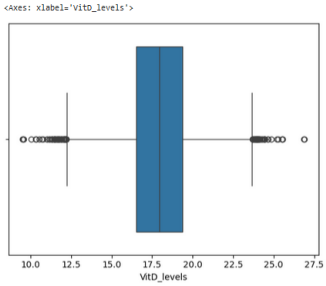


To looking like this, after I replaced the outliers:

```
[199]: plt.hist(med['VitD_levels'])
[199]: (array([  14.,  115.,  576., 1740., 3292., 2427., 1368.,  398.,   63.,
          7.]),
 array([ 9.51901164, 11.25793403, 12.99685643, 14.73577882, 16.47470121,
        18.21362361, 19.952546  , 21.6914684 , 23.43039079, 25.16931319,
        26.90823558]),
 <BarContainer object of 10 artists>)
```




```
[197]: sns.boxplot(x = 'VitD_levels', data= med)
[197]: <Axes: xlabel='VitD_levels'>
```



Here you can see all the necessary variables being labeled encoded:

[8]:
```
#Setting up label_encoder to help re-express the variables using just Yes / No
label_encoder = preprocessing.LabelEncoder()
```

[9]:
```
#Cheching it using just Yes / No
med['ReAdmis'].unique()
```

[9]:
```
array(['No', 'Yes'], dtype=object)
```

[10]:
```
#Encoding the lables with 0 and 1
med['ReAdmis']= label_encoder.fit_transform(med['ReAdmis'])
#hecking to make sure that it worked
med['ReAdmis'].unique()
```

[10]:
```
array([0, 1])
```

[11]:
```
#repeating the previous steps
med['HighBlood'].unique()
```

[11]:
```
array(['Yes', 'No'], dtype=object)
```

[12]:
```
med['HighBlood']= label_encoder.fit_transform(med['HighBlood'])
med['HighBlood'].unique()
```

[12]:
```
array([1, 0])
```

[13]:
```
med['Stroke'].unique()
```

[13]:
```
array(['No', 'Yes'], dtype=object)
```

[14]:
```
med['Stroke']= label_encoder.fit_transform(med['Stroke'])
med['Stroke'].unique()
```

[14]:
```
array([0, 1])
```

[15]:
```
med['Arthritis'].unique()
```

[15]:
```
array(['Yes', 'No'], dtype=object)
```

[16]:
```
med['Arthritis']= label_encoder.fit_transform(med['Arthritis'])
med['Arthritis'].unique()
```

[16]:
```
array([1, 0])
```

[17]:
```
med['Diabetes'].unique()
```

[17]:
```
array(['Yes', 'No'], dtype=object)
```

[18]:
```
med['Diabetes']= label_encoder.fit_transform(med['Diabetes'])
med['Diabetes'].unique()
```

[18]:
```
array([1, 0])
```

[19]:
```
med['Hyperlipidemia'].unique()
```

[19]:
```
array(['No', 'Yes'], dtype=object)
```

[20]:
```
med['Hyperlipidemia']= label_encoder.fit_transform(med['Hyperlipidemia'])
med['Hyperlipidemia'].unique()
```

[20]:
```
array([0, 1])
```

[21]:
```
med['BackPain'].unique()
```

[21]:
```
array(['Yes', 'No'], dtype=object)
```

[22]:
```
med['BackPain']= label_encoder.fit_transform(med['BackPain'])
med['BackPain'].unique()
```

[22]:
```
array([1, 0])
```

[23]:
```
med['Allergic_rhinitis'].unique()
```

[23]:
```
array(['Yes', 'No'], dtype=object)
```

[24]:
```
med['Allergic_rhinitis']= label_encoder.fit_transform(med['Allergic_rhinitis'])
med['Allergic_rhinitis'].unique()
```

[24]:
```
array([1, 0])
```

[25]:
```
med['Reflux_esophagitis'].unique()
```

[25]:
```
array(['No', 'Yes'], dtype=object)
```

[26]:
```
med['Reflux_esophagitis']= label_encoder.fit_transform(med['Reflux_esophagitis'])
med['Reflux_esophagitis'].unique()
```

[26]:
```
array([0, 1])
```

[27]:
```
med['Asthma'].unique()
```

[27]:
```
array(['Yes', 'No'], dtype=object)
```

[28]:
```
med['Asthma']= label_encoder.fit_transform(med['Asthma'])
med['Asthma'].unique()
```

[28]:
```
array([1, 0])
```

Finally here is the output when I renamed the item variables:

```
#Want to remane the last variables to make them more understandable
med.rename(columns = {'Item1':' TimelyAdmission',
                      'Item2':'TimelyTreatment',
                      'Item3':'TimelyVisits',
                      'Item4':'Reliability',
                      'Item5':'Options',
                      'Item6':'HoursOfTreatment',
                      'Item7':'CourteousStaff',
                      'Item8':'ActiveListening'},inplace=True)
```

```
#checking to make sure that it worked
med.head()
```

ID	City	State	County	Zip	Lat	...	TotalCharge	Additional_charges	Timely admission	Timely treatment	Timely visits	Reliability	Options	Hours of treatment	Courteous staff	Active Listening
32	Eva	AL	Morgan	35621	34.34960	...	3191.048774	17939.403420	3	3	2	2	4	3	3	4
95	Marianna	FL	Jackson	32446	30.84513	...	4214.905346	17612.998120	3	4	3	4	4	4	3	3
c9	Sioux Falls	SD	Minnehaha	57110	43.54321	...	2177.586768	17505.192460	2	4	4	4	3	4	3	3
07	New Richland	MN	Waseca	56072	43.89744	...	2465.118965	12993.437350	3	5	5	3	4	5	5	5
4a	West Point	VA	King William	23181	37.59894	...	1885.655137	3716.525786	2	1	3	3	5	3	4	3

C4: Limitations

There were two major limitations in my data cleaning process. The first limitation comes from the outliers. Since I was just given the raw data from the hospital with little knowledge about some of the variables, it was hard to know which of the outliers were unacceptable or unexpected. The second limitation is filling in large amount of numerical missing values and outliers with the average or median of the variable.

C5: Impact of Limitations

The outlier's limitations forced me to decide if the outliers were unacceptable or unexpected. To the best of my knowledge, most of the variable's outliers were acceptable and expected so I kept them in. This means that the data may be skewed in some way. Since there was a relatively small dataset it made it hard to delete all the nulls and outliers. This means I needed to fill these with the corresponding variables with their average or median. This will now over-represent those values in any analysis.

D1: Mitigation Code

This file is alongside this one with the name "PA_Tomanek_D206.ipynb"

D2: Clean Data

medical_clean_data.csv

E1: Principal Components

For my Principal Component Analysis (PCA) I used 6 variables including Income, VitD_levels, VitD_supp, Initial_days, TotalCharge, and Additional_charges. I used these variables because they are numerical and continuous, which is needed for PCA.

```
#Getting the Loading
loadings = pd.DataFrame(pca.components_.T,
                        columns = ['PC1', 'PC2', 'PC3', 'PC4', 'PC5', 'PC6', 'PC7','PC8'],
                        index=medical.columns)
loadings
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	PC8
Age	0.568709	-0.419065	-0.010955	0.025135	-0.009239	0.003185	-0.338216	0.621061
Income	-0.012175	-0.005723	0.678677	0.122323	-0.128924	0.712440	-0.004754	0.006148
VitD_levels	0.036988	0.016129	-0.180019	-0.895573	0.176535	0.357739	-0.064649	-0.024332
Doc_visits	0.001284	-0.020335	0.537372	-0.419244	-0.502882	-0.531138	-0.006475	0.003266
VitD_supp	0.042855	0.030675	0.466891	-0.041406	0.834163	-0.285771	0.000170	0.005335
Initial_days	0.407031	0.576041	0.006941	0.064355	-0.045958	-0.015737	-0.623807	-0.326829
TotalCharge	0.429578	0.560555	-0.007914	-0.024232	-0.035328	0.018515	0.626528	0.326285
Additional_charges	0.568335	-0.420346	0.001808	0.012049	-0.007152	0.008817	0.315742	-0.632720

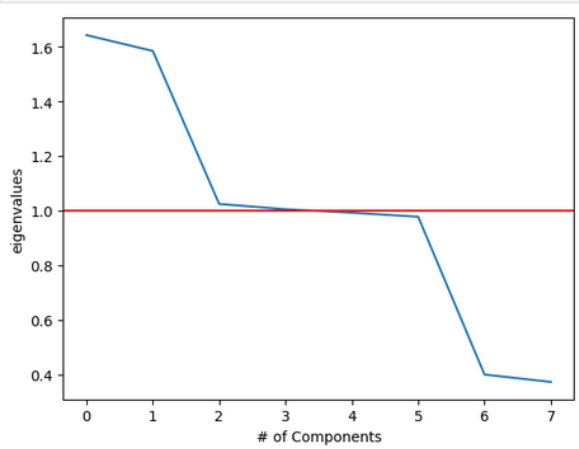
E2: Criteria Used

The most meaningful PCs to keep for business would be PC1, PC2, and PC3. This is because they have eigenvalues greater or equal to 1.

```
[100]: #Selecting PCs
       cov_matrix = np.dot(med_normalized.T, med_normalized)/medical.shape[0]

[101]: eigenvalues = [np.dot(eigenvector.T, np.dot(cov_matrix, eigenvector)) for eigenvector in pca.components_]

[102]: plt.plot(eigenvalues)
       plt.xlabel('# of Components')
       plt.ylabel('eigenvalues')
       plt.axhline(y=1, color="red")
       plt.show()
```



E3: Benefits

With the PCA the hospital can now see which variables have strong correlations with each other. This can help explain important things like why some patients may be staying at the hospital longer than others or why they may need to come back within a week. The hospital can then help inform and prepare the patients for what could happen to them.

F: Video

https://wgu.hosted.panopto.com/Panopto/Pages/Viewer.aspx?id=880ce33b-a736-4533-82a0-b1cd005b3e50

G: Sources of Third-Party Code

GeeksforGeeks. (2023, April 18). *Label encoding in Python*. https://www.geeksforgeeks.org/ml-label-encoding-of-datasets-in-python/#

Rename columns in a python pandas DataFrame. Sentry. (n.d.). https://sentry.io/answers/rename-columns-in-a-python-pandas-dataframe/

H: Sources

N/A

I: Professional Communication